

UNIVERSIDAD DE SAN CARLOS DE GUATEMALA
FACULTAD DE INGENIERÍA
ESCUELA DE CIENCIAS Y SISTEMAS
SISTEMAS OPERATIVOS 1 SECCIÓN N
ING. SERGIO MÉNDEZ
AUX. ALLEN ROMÁN
VACACIONES DEL SEGUNDO SEMESTRE 2024



PROYECTO 2

Sistema de Asignaciones de Cursos

Objetivos

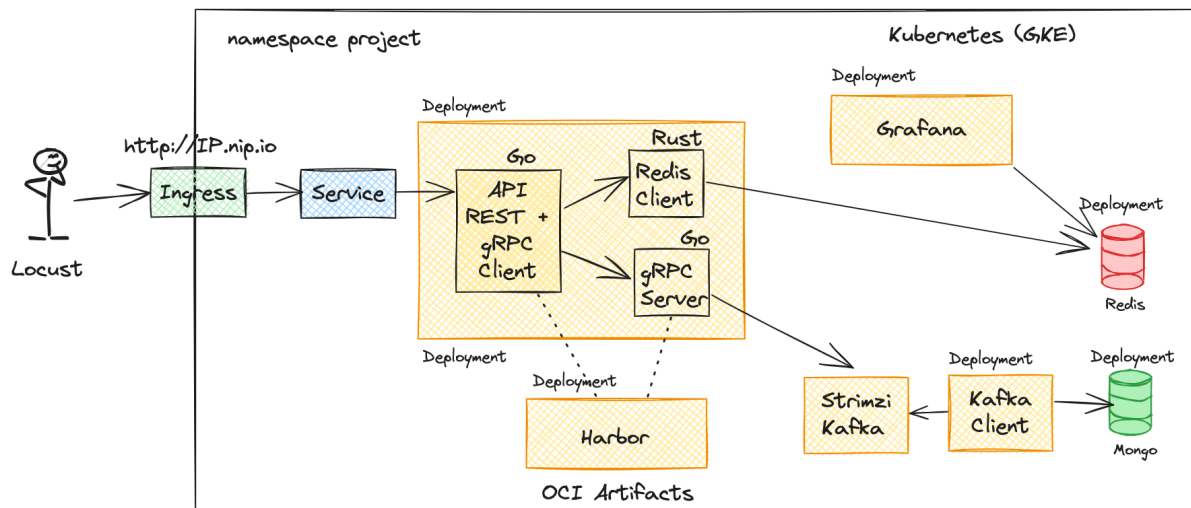
- Utilizar Kubernetes como plataforma para crear sistemas distribuidos
- Utilizar estándar OCI para almacenamiento para imágenes en containers como para almacenamiento de archivos para carga de información
- Utilizar bases de datos NoSQL (Mongo, Redis) para manejo masivo de datos
- Manejar procesamiento masivo de información utilizando colas de mensajes con Kafka
- Visualizar información en tiempo real de la información

Introducción

El objetivo de este proyecto es realizar una infraestructura que soporte procesamiento de información masiva y la muestre en tiempo real. Para ello se utilizará Kubernetes como plataforma para crear este sistema distribuido y NoSQL (Redis y Mongo) para el procesamiento de información. Y Grafana como visualizador de información en tiempo real. En este caso este proyecto consiste en simular las asignaciones de cursos en horas pico en cuyo caso los sistemas de asignación de saturación. Para este proyecto se simulará la asignación de cursos para las distintas carreras y regiones de Guatemala para la Universidad de San Carlos de Guatemala, asumiendo que se tuviera el mismo sistema y fechas de asignaciones, simulando así un sistema saturado en las últimas horas de asignaciones. El objetivo final no es hacer un sistema exacto de asignaciones sino más bien simular el manejo masivo de información y el cálculo de analíticas en tiempo real como lo son en sistemas de votaciones o redes sociales donde se procesan mensajes.

Arquitectura

La arquitectura para este sistema se muestra en el siguiente diagrama:



Generador de Tráfico y Carga de Información

Para comenzar se simula la generación de información de asignaciones con la creación de un generador de tráfico de entrada utilizando **Locust** y **Python** como lenguaje de programación. El tráfico será recibido en un Ingress y generar los datos de asignación de cursos directamente en el script de Locust, se debe configurar Locust para que consuma un archivo de entrada almacenado como un artefacto OCI (Open Container Initiative) utilizando el comando **oras** (<https://oras.land/>). Este archivo contendrá los datos de prueba en formato JSON y será almacenado en un registro de contenedores, Harbor, para su distribución.

Configuración de Locust

Se debe de generar un scripts de prueba Locust para simular el comportamiento de usuarios ingresando calificaciones. El JSON de Prueba tendrá un formato de entrada similar al siguiente:

```
{
  "curso": "SO1",
  "facultad": "ingenieria",
  "carrera": "sistemas",
  "region": "NORTE"
}
```

Los Cursos a Monitorear pueden ser por ejemplo:

- **SO1** - Sistemas Operativos 1
- **ANP** - Análisis Probabilístico
- **IO2** - Investigación de operaciones 2
- **etc**

Las Facultades a Monitorear pueden ser por ejemplo: Ingeniería, Medicina, etc.

Las Carreras a Monitorear pueden ser por ejemplo: Sistemas, Industrial, Diseño Gráfico, etc.

Las Regiones a Monitorear pueden ser por ejemplo:

- **METROPOLITANA:** Guatemala
- **NORTE:** Baja Verapaz y Alta Verapaz
- **NORORIENTAL:** El Progreso, Izabal, Zacapa y Chiquimula
- **SURORIENTAL:** Santa Rosa, Jalapa y Jutiapa

- **CENTRAL:** Sacatepéquez, Chimaltenango y Escuintla
- **SUROCCIDENTAL:** Sololá, Totonicapán, Quetzaltenango, Suchitepéquez, Retalhuleu y San Marcos
- **NOROCCIDENTAL:** Huehuetenango y Quiché
- **PETEN:** Petén

Esta información servirá para los reportes a mostrar en tiempo real.

Nota: Locust debe de enviar información masiva y concurrente, teniendo cargas entre 3-10k datos.

Kubernetes

Se encargará de la orquestación de los contenedores de las diferentes partes de la aplicación. Se debe de crear un Namespace llamado: **project**, y trabajar dentro de él para crear los **PODS/DEPLOYMENTS** y sus respectivos **SERVICES E INGRESS**.

Ingress

Se tiene un Ingress que sirve como punto de entrada para captura de información, el cuál redirige su tráfico a un servicio apuntando a un deployment que procesa la información, esté implementando una API Rest en Go, y otras funcionalidades usando gRPC para procesar la información. Este punto de entrada debe de estar configurado usando NIP para tener un DNS temporal para acceder a servicio que captura e inserta datos.

Harbor

Harbor será utilizado como un registro de contenedores privado para almacenar las imágenes Docker necesarias para el despliegue del sistema. Se debe configurar un proyecto dentro de Harbor con el nombre project y subir las imágenes correspondientes a los servicios desarrollados (gRPC en Golang, API en Rust, y cualquier otro componente necesario). Además, se debe asegurar que las imágenes sean accesibles desde el clúster de Kubernetes configurando las credenciales de acceso en el clúster mediante un Secret. Esto garantizará que los Pods puedan extraer las imágenes de manera segura y eficiente.

Deployments

Nuestro sistema de asignaciones masiva cuenta con 2 deployments, los cuales capturan la información, llenan las colas, las procesan y las visualiza. A continuación explicamos estos deployments:

1) d-api-rest-grpc: Captura de información y llenado de colas

Después que nuestro Ingress captura la información la redirige a un Deployment llamado process que tiene 3 containers los cuales tienen las siguientes funcionalidades:

container1(API Rest+gRPC Client):

Este container implementa una API Rest en Go, que se encarga de recibir la información, una vez obtenida la información de un Post a la API se utiliza gRPC implementado como cliente para consumir la función implementada en el siguiente container de gRPC como servidor.

container2(gRPC Server):

Este container atiende la solicitudes solicitadas por el container1 y el código cliente de gRPC para enviar la información a una cola de mensajes en Kafka que al momento de acumularse las solicitudes lo irá procesando asíncronamente hasta vaciar toda la cola con información pendiente de procesar.

container3(Rust):

Este container Recibe información del container 1, y almacena información en Redis que permita mostrar la siguiente información:

- Asignados por Región
- Total Asignados en Guatemala
- Total Asignados por Carrera
- Total Asignados por Facultad
- Total Asignados por Curso

2) d-kafka-process: Procesamiento de colas (Confluent librerías)

Este deployment contiene un solo container que se encarga de leer la cola de Kafka llenada por el deployment d-api-rest-grpc para luego consumir cada mensaje con información a ser insertada a Mongo como Log.

Código en: <https://github.com/sergioarmqpl/CloudNativeNow2023>

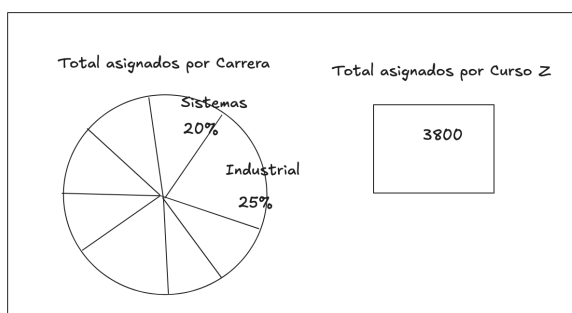
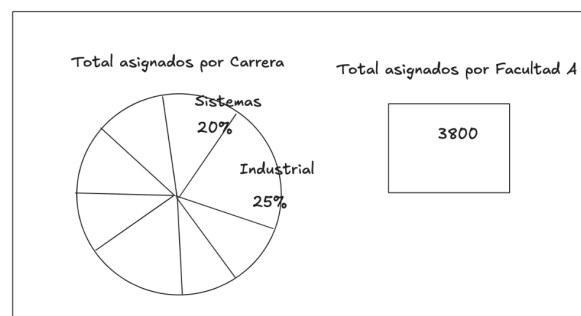
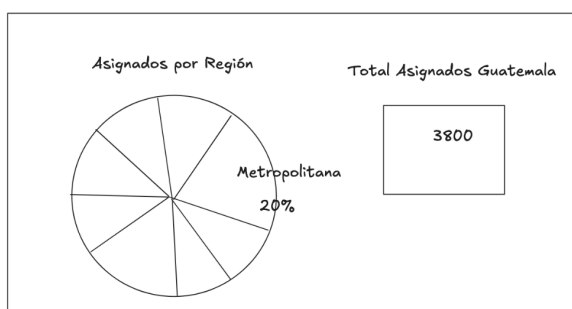
Instalar Kafka con Strimzi <https://strimzi.io/> y usar la guía <https://strimzi.io/quickstarts/>

3) Grafana

Se debe crear un sitio web para mostrar en tiempo real los datos cargados, para su desarrollo, este debe ser un Dashboard y mostrar lo siguiente:

- Total de asignados en Guatemala
- Cursos asignados en una carrera
- Cursos asignados por región
- Asignados por curso sin importar región o carrera

Estos datos deben de ser mostrados como contadores y gráficas de Pie. A continuación se muestran las 3 gráficas a mostrar en tiempo real:



Estas gráficas tendrán que tener combo box con los cursos, para ir cambiando las gráficas según los cursos, facultades y regiones ingresadas.

Link sugerido para instalar Grafana:

<https://github.com/PacktPublishing/Edge-Computing-Systems-with-Kubernetes/blob/main/ch11/yaml/grafana-deployment.yaml>

Autoscaling de k8s (Horizontal Pod Autoscaling)

El HPA ajusta automáticamente la cantidad de PODS en ejecución en un despliegue o conjunto de réplicas según la utilización de la CPU u otras métricas. Para este proyecto cada Deployment debe tener un mínimo de 1 réplica y un máximo de 3 réplicas. Además, se debe configurar que el uso de la CPU no exceda el 50%.

Bases de datos NoSQL

Estas almacenan la información masiva a mostrar en tiempo real, y el enlace sugerido para instalar es:

- <https://github.com/PacktPublishing/Edge-Computing-Systems-with-Kubernetes/tree/main/ch10/yaml>

Requisitos de entrega

Github

- El código fuente debe de ser gestionado por un repositorio privado de Github
- Nombre de la carpeta en el repositorio de tareas: **proyecto2**
- Agregar a los auxiliares al repositorio de GitHub: **Allenrovas**

Calificación

- Al momento de la calificación se verificará la última versión publicada en el repositorio de GitHub
- Cualquier copia parcial o total tendrán nota de 0 puntos y serán reportadas al catedrático y a la Escuela de Ciencias y Sistemas.

Entregables

- La entrega se debe realizar antes de las 23:59 del 2 de enero de 2024.
- La forma de entrega es mediante UEDI subiendo el enlace del repositorio.